Docling Document

This is an automatic generated API reference of the DoclingDocument type.

`mod` doc

Package for models defined by the Document type.

Classes:

- `DoclingDocument` – DoclingDocument.
- `DocumentOrigin` – FileSource.
- `DocItem` – Base type for any element that carries content, can be a leaf node.
- `DocItemLabel` – DocItemLabel.
- `ProvenanceItem` – Provenance information for elements extracted from a textual document.
- `GroupItem` – GroupItem.
- `GroupLabel` – GroupLabel.
- `NodeItem` – NodeItem.
- `PageItem` – PageItem.
- `FloatingItem` – FloatingItem.
- `TextItem` – TextItem.
- `TableItem` – TableItem.
- `TableCell` – TableCell.
- `TableData` – BaseTableData.
- `TableCellLabel` – TableCellLabel.
- `KeyValueItem` – KeyValueItem.
- `SectionHeaderItem` – SectionItem.
- `PictureItem` – PictureItem.
- `ImageRef` – ImageRef.
- `PictureClassificationClass` – PictureClassificationData.
- `PictureClassificationData` – PictureClassificationData.
- `RefItem` – RefItem.
- `BoundingBox` – BoundingBox.
- `CoordOrigin` – CoordOrigin.
- `ImageRefMode` – ImageRefMode.

- `Size` – Size.

`class` DoclingDocument

Bases: `BaseModel`

DoclingDocument.

Methods:

- `add_code` – add_code.
- `add_comment` – Adds a comment to the document, assigning it to the given targets.
- `add_document` – Adds the content from the body of a DoclingDocument to this document under a specific parent.
- `add_form` – add_form.
- `add_formula` – add_formula.
- `add_group` – add_group.
- `add_heading` – add_heading.
- `add_inline_group` – add_inline_group.
- `add_key_values` – add_key_values.
- `add_list_group` – add_list_group.
- `add_list_item` – add_list_item.
- `add_node_items` – Adds multiple NodeItems and their children under a parent in this document.
- `add_ordered_list` – add_ordered_list.
- `add_page` – add_page.
- `add_picture` – add_picture.
- `add_table` – add_table.
- `add_table_cell` – Add a table cell to the table.
- `add_text` – add_text.
- `add_title` – add_title.
- `add_unordered_list` – add_unordered_list.
- `append_child_item` – Adds an item.
- `check_version_is_compatible` – Check if this document version is compatible with SDK schema version.
- `concatenate` – Concatenate multiple documents into a single document.
- `delete_items` – Deletes an item, given its instance or ref, and any children it has.
- `delete_items_range` – Deletes all NodeItems and their children in the range from the start NodeItem to the end NodeItem.
- `export_to_dict` – Export to dict.
- `export_to_doctags` – Exports the document content to a DocumentToken format.
- `export_to_document_tokens` – Export to DocTags format.
- `export_to_element_tree` – Export_to_element_tree.
- `export_to_html` – Serialize to HTML.

- `export_to_markdown` – Serialize to Markdown.
- `export_to_text` – Export to plain text.
- `export_to_vtt` – Serializes the Docling document to WebVTT format.
- `extract_items_range` – Extracts NodeItems and children in the range from the start NodeItem to the end as a new DoclingDocument.
- `filter` – Create a new document based on the provided filter parameters.
- `get_visualization` – Get visualization of the document as images by page.
- `insert_code` – Creates a new CodeItem item and inserts it into the document.
- `insert_document` – Inserts the content from the body of a DoclingDocument into this document at a specific position.
- `insert_form` – Creates a new FormItem item and inserts it into the document.
- `insert_formula` – Creates a new FormulaItem item and inserts it into the document.
- `insert_group` – Creates a new GroupItem item and inserts it into the document.
- `insert_heading` – Creates a new SectionHeaderItem item and inserts it into the document.
- `insert_inline_group` – Creates a new InlineGroup item and inserts it into the document.
- `insert_item_after_sibling` – Inserts an item, given its node_item instance, after other as a sibling.
- `insert_item_before_sibling` – Inserts an item, given its node_item instance, before other as a sibling.
- `insert_key_values` – Creates a new KeyValueItem item and inserts it into the document.
- `insert_list_group` – Creates a new ListGroup item and inserts it into the document.
- `insert_list_item` – Creates a new ListItem item and inserts it into the document.
- `insert_node_items` – Insert multiple NodeItems and their children at a specific position in the document.
- `insert_picture` – Creates a new PictureItem item and inserts it into the document.
- `insert_table` – Creates a new TableItem item and inserts it into the document.
- `insert_text` – Creates a new TextItem item and inserts it into the document.
- `insert_title` – Creates a new TitleItem item and inserts it into the document.
- `iterate_items` – Iterate elements with level.
- `load_from_doctags` – Load Docling document from lists of DocTags and Images.
- `load_from_json` – load_from_json.
- `load_from_yaml` – load_from_yaml.
- `num_pages` – num_pages.
- `print_element_tree` – Print_element_tree.
- `replace_item` – Replace item with new item.
- `save_as_doctags` – Save the document content to DocTags format.
- `save_as_document_tokens` – Save the document content to a DocumentToken format.
- `save_as_html` – Save to HTML.
- `save_as_json` – Save as json.
- `save_as_markdown` – Save to markdown.
- `save_as_vtt` – Saves the Docling document to a file in WebVTT format.

- `save_as_yaml` – Save as yaml.
- `transform_to_content_layer` – transform_to_content_layer.
- `validate_document` – validate_document.
- `validate_misplaced_list_items` – validate_misplaced_list_items.
- `validate_tree` – validate_tree.

Attributes:

- `body` (`GroupItem`) –
- `form_items` (`list[FormItem]`) –
- `furniture` (`Annotated[GroupItem, Field(deprecated=True)]`) –
- `groups` (`list[Union[ListGroup, InlineGroup, GroupItem]]`) –
- `key_value_items` (`list[KeyValueItem]`) –
- `name` (`str`) –
- `origin` (`Optional[DocumentOrigin]`) –
- `pages` (`dict[int, PageItem]`) –
- `pictures` (`list[PictureItem]`) –
- `schema_name` (`Literal['DoclingDocument']`) –
- `tables` (`list[TableItem]`) –
- `texts` (`list[Union[TitleItem, SectionHeaderItem, ListItem, CodeItem, FormulaItem, TextItem]]`) –
- `version` (`Annotated[str, StringConstraints(pattern=VERSION_PATTERN, strict=True)]`) –

attr **body**

```
body: GroupItem = GroupItem(name='_root_', self_ref='#/body')
```

attr **form_items**

```
form_items: list[FormItem] = []
```

attr **furniture**

```
furniture: Annotated[GroupItem, Field(deprecated=True)] = GroupItem(name='_root_', self_ref='#/furniture',
content_layer=FURNITURE)
```

attr **groups**

```
groups: list[Union[ListGroup, InlineGroup, GroupItem]] = []
```

attr **key_value_items**

```
key_value_items: list[KeyValueItem] = []
```

attr **name**

```
name: str
```

attr origin

```
origin: Optional[DocumentOrigin] = None
```

attr pages

```
pages: dict[int, PageItem] = {}
```

attr pictures

```
pictures: list[PictureItem] = []
```

attr schema_name

```
schema_name: Literal['DoclingDocument'] = 'DoclingDocument'
```

attr tables

```
tables: list[TableItem] = []
```

attr texts

```
texts: list[Union[TitleItem, SectionHeaderItem, ListItem, CodeItem, FormulaItem, TextItem]] = []
```

attr version

```
version: Annotated[str, StringConstraints(pattern=VERSION_PATTERN, strict=True)] = CURRENT_VERSION
```

meth add_code

```
add_code(text: str, code_language: Optional[CodeLanguageLabel] = None, orig: Optional[str] = None, caption: Optional[Union[TextItem, RefItem]] = None, prov: Optional[ProvenanceItem] = None, parent: Optional[NodeItem] = None, content_layer: Optional[ContentLayer] = None, formatting: Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl, Path]] = None)
```

add_code.

Parameters:

- **text** (`str`) – str:
- **code_language** (`Optional[CodeLanguageLabel]`, default: `None`) – Optional[CodeLanguageLabel]: (Default value = None)
- **orig** (`Optional[str]`, default: `None`) – Optional[str]: (Default value = None)
- **caption** (`Optional[Union[TextItem, RefItem]]`, default: `None`) – Optional[Union[TextItem:
- **RefItem]]** – (Default value = None)
- **prov** (`Optional[ProvenanceItem]`, default: `None`) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (`Optional[NodeItem]`, default: `None`) – Optional[NodeItem]: (Default value = None)

meth **add_comment**

```
add_comment(*, text: str, prov: Optional[ProvenanceItem] = None, parent: Optional[NodeItem] = None, targets:
Optional[list[Union[DocItem, tuple[DocItem, tuple[int, int]]]]] = None)
```

Adds a comment to the document, assigning it to the given targets.

Parameters:

- **text** (`str`) – str:
- **prov** (`Optional[ProvenanceItem]`, default: `None`) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (`Optional[NodeItem]`, default: `None`) – Optional[NodeItem]: (Default value = None)
- **targets** (`Optional[list[Union[DocItem, tuple[DocItem, tuple[int, int]]]]]`, default: `None`) – list[Union[DocItem, tuple[DocItem, tuple[int, int]]]]: (Default value = None) Each list element can be either a single DocItem or a tuple of a DocItem and a span range (start_inclusive, end_exclusive).

meth **add_document**

```
add_document(doc: DoclingDocument, parent: Optional[NodeItem] = None) -> None
```

Adds the content from the body of a DoclingDocument to this document under a specific parent.

Parameters:

- **doc** (`DoclingDocument`) – DoclingDocument: The document whose content will be added
- **parent** (`Optional[NodeItem]`, default: `None`) – Optional[NodeItem]: The parent NodeItem under which new items are added (Default value = None)

Returns:

- `None` – None

meth **add_form**

```
add_form(graph: GraphData, prov: Optional[ProvenanceItem] = None, parent: Optional[NodeItem] = None)
```

add_form.

Parameters:

- **graph** (`GraphData`) – GraphData:
- **prov** (`Optional[ProvenanceItem]`, default: `None`) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (`Optional[NodeItem]`, default: `None`) – Optional[NodeItem]: (Default value = None)

meth **add_formula**

```
add_formula(text: str, orig: Optional[str] = None, prov: Optional[ProvenanceItem] = None, parent:
Optional[NodeItem] = None, content_layer: Optional[ContentLayer] = None, formatting: Optional[Formatting] =
None, hyperlink: Optional[Union[AnyUrl, Path]] = None)
```

add_formula.

Parameters:

- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **level** – LevelNumber: (Default value = 1)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth add_group

```
add_group(label: Optional[GroupLabel] = None, name: Optional[str] = None, parent: Optional[NodeItem] = None,
content_layer: Optional[ContentLayer] = None) -> GroupItem
```

add_group.

Parameters:

- **label** (Optional[GroupLabel], default: None) – Optional[GroupLabel]: (Default value = None)
- **name** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth add_heading

```
add_heading(text: str, orig: Optional[str] = None, level: LevelNumber = 1, prov: Optional[ProvenanceItem] =
None, parent: Optional[NodeItem] = None, content_layer: Optional[ContentLayer] = None, formatting:
Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl, Path]] = None)
```

add_heading.

Parameters:

- **label** – DocItemLabel:
- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **level** (LevelNumber, default: 1) – LevelNumber: (Default value = 1)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth add_inline_group

```
add_inline_group(name: Optional[str] = None, parent: Optional[NodeItem] = None, content_layer:
Optional[ContentLayer] = None) -> InlineGroup
```

add_inline_group.

meth add_key_values

```
add_key_values(graph: GraphData, prov: Optional[ProvenanceItem] = None, parent: Optional[NodeItem] = None)
```

add_key_values.

Parameters:

- **graph** (GraphData) – GraphData:
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth **add_list_group**

```
add_list_group(name: Optional[str] = None, parent: Optional NodeItem = None, content_layer:
Optional[ContentLayer] = None) -> ListGroup
```

add_list_group.

meth **add_list_item**

```
add_list_item(text: str, enumerated: bool = False, marker: Optional[str] = None, orig: Optional[str] = None,
prov: Optional[ProvenanceItem] = None, parent: Optional NodeItem = None, content_layer:
Optional[ContentLayer] = None, formatting: Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl,
Path]] = None)
```

add_list_item.

Parameters:

- **label** – str:
- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth **add_node_items**

```
add_node_items(node_items: list[NodeItem], doc: DoclingDocument, parent: Optional NodeItem = None) -> None
```

Adds multiple NodeItems and their children under a parent in this document.

Parameters:

- **node_items** (list[NodeItem]) – list[NodeItem]: The NodeItems to be added
- **doc** (DoclingDocument) – DoclingDocument: The document to which the NodeItems and their children belong
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: The parent NodeItem under which new items are added (Default value = None)

Returns:

- None – None

meth **add_ordered_list**

```
add_ordered_list(name: Optional[str] = None, parent: Optional NodeItem = None, content_layer:
Optional[ContentLayer] = None) -> GroupItem
```

add_ordered_list.

meth **add_page**

```
add_page(page_no: int, size: Size, image: Optional[ImageRef] = None) -> PageItem
```

add_page.

Parameters:

- **page_no** (*int*) – int:
- **size** (*Size*) – Size:

meth **add_picture**

```
add_picture(annotations: Optional[list[PictureDataType]] = None, image: Optional[ImageRef] = None, caption:
Optional[Union[TextItem, RefItem]] = None, prov: Optional[ProvenanceItem] = None, parent: Optional[NodeItem]
= None, content_layer: Optional[ContentLayer] = None)
```

add_picture.

Parameters:

- **data** – Optional[list[PictureData]]: (Default value = None)
- **caption** (*Optional[Union[TextItem, RefItem]]*, default: *None*) – Optional[Union[TextItem:
- **RefItem]]** – (Default value = None)
- **prov** (*Optional[ProvenanceItem]*, default: *None*) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (*Optional[NodeItem]*, default: *None*) – Optional[NodeItem]: (Default value = None)

meth **add_table**

```
add_table(data: TableData, caption: Optional[Union[TextItem, RefItem]] = None, prov: Optional[ProvenanceItem]
= None, parent: Optional[NodeItem] = None, label: DocItemLabel = TABLE, content_layer: Optional[ContentLayer]
= None, annotations: Optional[list[TableAnnotationType]] = None)
```

add_table.

Parameters:

- **data** (*TableData*) – TableData:
- **caption** (*Optional[Union[TextItem, RefItem]]*, default: *None*) – Optional[Union[TextItem, RefItem]]: (Default value = None)
- **prov** (*Optional[ProvenanceItem]*, default: *None*) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (*Optional[NodeItem]*, default: *None*) – Optional[NodeItem]: (Default value = None)
- **label** (*DocItemLabel*, default: *TABLE*) – DocItemLabel: (Default value = DocItemLabel.TABLE)

meth **add_table_cell**

```
add_table_cell(table_item: TableItem, cell: TableCell) -> None
```

Add a table cell to the table.

meth **add_text**

```
add_text(label: DocItemLabel, text: str, orig: Optional[str] = None, prov: Optional ProvenanceItem = None,
parent: Optional[NodeItem] = None, content_layer: Optional[ContentLayer] = None, formatting:
Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl, Path]] = None, *, source: Optional[SourceType]
= None)
```

add_text.

Parameters:

- **label** (DocItemLabel) – str:
- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth **add_title**

```
add_title(text: str, orig: Optional[str] = None, prov: Optional ProvenanceItem = None, parent:
Optional[NodeItem] = None, content_layer: Optional[ContentLayer] = None, formatting: Optional[Formatting] =
None, hyperlink: Optional[Union[AnyUrl, Path]] = None)
```

add_title.

Parameters:

- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **level** – LevelNumber: (Default value = 1)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **parent** (Optional[NodeItem], default: None) – Optional[NodeItem]: (Default value = None)

meth **add_unordered_list**

```
add_unordered_list(name: Optional[str] = None, parent: Optional NodeItem = None, content_layer:
Optional[ContentLayer] = None) -> GroupItem
```

add_unordered_list.

meth **append_child_item**

```
append_child_item(*, child: NodeItem, parent: Optional NodeItem = None) -> None
```

Adds an item.

meth **check_version_is_compatible**

```
check_version_is_compatible(v: str) -> str
```

Check if this document version is compatible with SDK schema version.

meth **concatenate**

```
concatenate(docs: Sequence[DoclingDocument]) -> DoclingDocument
```

Concatenate multiple documents into a single document.

meth **delete_items**

```
delete_items(*, node_items: list[NodeItem]) -> None
```

Deletes an item, given its instance or ref, and any children it has.

meth **delete_items_range**

```
delete_items_range(*, start: NodeItem, end: NodeItem, start_inclusive: bool = True, end_inclusive: bool = True) -> None
```

Deletes all NodeItems and their children in the range from the start NodeItem to the end NodeItem.

Parameters:

- **start** (`NodeItem`) – NodeItem: The starting NodeItem of the range
- **end** (`NodeItem`) – NodeItem: The ending NodeItem of the range
- **start_inclusive** (`bool`, default: `True`) – bool: (Default value = True): If True, the start NodeItem will also be deleted
- **end_inclusive** (`bool`, default: `True`) – bool: (Default value = True): If True, the end NodeItem will also be deleted

Returns:

- `None` – None

meth **export_to_dict**

```
export_to_dict(mode: str = 'json', by_alias: bool = True, exclude_none: bool = True, coord_precision: Optional[int] = None, confid_precision: Optional[int] = None) -> dict[str, Any]
```

Export to dict.

meth **export_to_doctags**

```
export_to_doctags(delim: str = '', from_element: int = 0, to_element: int = maxsize, labels: Optional[set[DocItemLabel]] = None, xsize: int = 500, ysize: int = 500, add_location: bool = True, add_content: bool = True, add_page_index: bool = True, add_table_cell_location: bool = False, add_table_cell_text: bool = True, minified: bool = False, pages: Optional[set[int]] = None) -> str
```

Exports the document content to a DocumentToken format.

Operates on a slice of the document's body as defined through arguments from_element and to_element; defaulting to the whole main_text.

Parameters:

- **delim** (`str`, default: `''`) – str: (Default value = "") Deprecated
- **from_element** (`int`, default: `0`) – int: (Default value = 0)

- **to_element** (int, default: maxsize) – Optional[int]: (Default value = None)
- **labels** (Optional[set[DocItemLabel]], default: None) – set[DocItemLabel]
- **xsize** (int, default: 500) – int: (Default value = 500)
- **ysize** (int, default: 500) – int: (Default value = 500)
- **add_location** (bool, default: True) – bool: (Default value = True)
- **add_content** (bool, default: True) – bool: (Default value = True)
- **add_page_index** (bool, default: True) – bool: (Default value = True)
- **flagsadd_table_cell_location** – bool
- **add_table_cell_text** (bool, default: True) – bool: (Default value = True)
- **minified** (bool, default: False) – bool: (Default value = False)
- **pages** (Optional[set[int]], default: None) – set[int]: (Default value = None)

Returns:

- **str** – The content of the document formatted as a DocTags string.

meth export_to_document_tokens

```
export_to_document_tokens(*args, **kwargs)
```

Export to DocTags format.

meth export_to_element_tree

```
export_to_element_tree() -> str
```

Export_to_element_tree.

meth export_to_html

```
export_to_html(from_element: int = 0, to_element: int = maxsize, labels: Optional[set[DocItemLabel]] = None,
enable_chart_tables: bool = True, image_mode: ImageRefMode = PLACEHOLDER, formula_to_mathml: bool = True,
page_no: Optional[int] = None, html_lang: str = 'en', html_head: str = 'null', included_content_layers:
Optional[set[ContentLayer]] = None, split_page_view: bool = False, include_annotations: bool = True) -> str
```

Serialize to HTML.

meth export_to_markdown

```
export_to_markdown(delim: str = '\n\n', from_element: int = 0, to_element: int = maxsize, labels:
Optional[set[DocItemLabel]] = None, strict_text: bool = False, escape_html: bool = True, escape_underscores:
bool = True, image_placeholder: str = '<!-- image -->', enable_chart_tables: bool = True, image_mode:
ImageRefMode = PLACEHOLDER, indent: int = 4, text_width: int = -1, page_no: Optional[int] = None,
included_content_layers: Optional[set[ContentLayer]] = None, page_break_placeholder: Optional[str] = None,
include_annotations: bool = True, mark_annotations: bool = False, compact_tables: bool = False, *,
use_legacy_annotations: Optional[bool] = None, allowed_meta_names: Optional[set[str]] = None,
blocked_meta_names: Optional[set[str]] = None, mark_meta: bool = False) -> str
```

Serialize to Markdown.

Operates on a slice of the document's body as defined through arguments from_element and to_element; defaulting to the whole document.

Parameters:

- **`delim`** (`str`, default: `'\n\n'`) – Deprecated.
- **`from_element`** (`int`, default: `0`) – Body slicing start index (inclusive). (Default value = 0).
- **`to_element`** (`int`, default: `maxsize`) – Body slicing stop index (exclusive). (Default value = maxint).
- **`labels`** (`Optional[set[DocItemLabel]]`, default: `None`) – The set of document labels to include in the export. None falls back to the system-defined default.
- **`strict_text`** (`bool`, default: `False`) – Deprecated.
- **`escape_html`** (`bool`, default: `True`) – bool: Whether to escape HTML reserved characters in the text content of the document. (Default value = True).
- **`escape_underscores`** (`bool`, default: `True`) – bool: Whether to escape underscores in the text content of the document. (Default value = True).
- **`image_placeholder`** (`str`, default: `'<!-- image -->'`) – The placeholder to include to position images in the markdown. (Default value = "\<!-- image -->").
- **`image_mode`** (`ImageRefMode`, default: `PLACEHOLDER`) – The mode to use for including images in the markdown. (Default value = ImageRefMode.PLACEHOLDER).
- **`indent`** (`int`, default: `4`) – The indent in spaces of the nested lists. (Default value = 4).
- **`included_content_layers`** (`Optional[set[ContentLayer]]`, default: `None`) – The set of layels to include in the export. None falls back to the system-defined default.
- **`page_break_placeholder`** (`Optional[str]`, default: `None`) – The placeholder to include for marking page breaks. None means no page break placeholder will be used.
- **`include_annotations`** (`bool`, default: `True`) – bool: Whether to include annotations in the export; only considered if item does not have meta. (Default value = True).
- **`mark_annotations`** (`bool`, default: `False`) – bool: Whether to mark annotations in the export; only considered if item does not have meta. (Default value = False).
- **`compact_tables`** (`bool`, default: `False`) – bool: Whether to use compact table format without column padding. (Default value = False).
- **`use_legacy_annotations`** (`Optional[bool]`, default: `None`) – bool: Deprecated; legacy annotations considered only when meta not present.
- **`mark_meta`** (`bool`, default: `False`) – bool: Whether to mark meta in the export
- **`allowed_meta_names`** (`Optional[set[str]]`, default: `None`) – Optional[set[str]]: Meta names to allow; None means all meta names are allowed.
- **`blocked_meta_names`** (`Optional[set[str]]`, default: `None`) – Optional[set[str]]: Meta names to block; takes precedence over allowed_meta_names.

Returns:

- `str` – The exported Markdown representation.

`meth` **export_to_text**

```
export_to_text(delim: str = '\n\n', from_element: int = 0, to_element: int = maxsize, labels:
Optional[set[DocItemLabel]] = None, page_no: Optional[int] = None, included_content_layers:
Optional[set[ContentLayer]] = None, page_break_placeholder: Optional[str] = None) -> str
```

Export to plain text.

Produces clean plain text without any Markdown decoration. Heading markers (`#`), bold/italic markers, and hyperlink syntax are all stripped. List bullets (`-`), ordered list numbers, and table-cell separators (`|`) are preserved as they aid readability.

Parameters:

- **`delim`** (`str`, default: `'\n\n'`) – Deprecated.
- **`from_element`** (`int`, default: `0`) – Body slicing start index (inclusive). (Default value = 0).
- **`to_element`** (`int`, default: `maxsize`) – Body slicing stop index (exclusive). (Default value = maxint).
- **`labels`** (`Optional[set[DocItemLabel]]`, default: `None`) – The set of document labels to include in the export. None falls back to the system-defined default.
- **`page_no`** (`Optional[int]`, default: `None`) – If set, only content from this page is exported.
- **`included_content_layers`** (`Optional[set[ContentLayer]]`, default: `None`) – The set of layers to include. None falls back to the system-defined default.
- **`page_break_placeholder`** (`Optional[str]`, default: `None`) – String inserted at page boundaries. None means no page-break marker is emitted.

Returns:

- `str` – The exported plain-text representation.

`meth` export_to_vtt

```
export_to_vtt(included_content_layers: set[ContentLayer] | None = None, omit_hours_if_zero: bool = False,
omit_voice_end: bool = False) -> str
```

Serializes the Docling document to WebVTT format.

Args: included_content_layers: The content layers to serializes. If ommitted, the `DEFAULT_CONTENT_LAYERS` will be serialized. omit_hours_if_zero: If True, omit hours when they are 0 in the timings. omit_voice_end: If True and cue blocks have a WebVTT cue voice span as the only component, omit the voice end tag for brevity.

Returns: A string representation of the Docling document in WebVTT format.

`meth` extract_items_range

```
extract_items_range(*, start: NodeItem, end: NodeItem, start_inclusive: bool = True, end_inclusive: bool =
True, delete: bool = False) -> DoclingDocument
```

Extracts NodeItems and children in the range from the start NodeItem to the end as a new DoclingDocument.

Parameters:

- **`start`** (`NodeItem`) – NodeItem: The starting NodeItem of the range (must be a direct child of the document body)
- **`end`** (`NodeItem`) – NodeItem: The ending NodeItem of the range (must be a direct child of the document body)
- **`start_inclusive`** (`bool`, default: `True`) – bool: (Default value = True): If True, the start NodeItem will also be extracted

- **`end_inclusive`** (`bool`, default: `True`) – bool: (Default value = True): If True, the end NodeItem will also be extracted
- **`delete`** (`bool`, default: `False`) – bool: (Default value = False): If True, extracted items are deleted in the original document

Returns:

- `DoclingDocument` – DoclingDocument: A new document containing the extracted NodeItems and their children

`meth` filter

```
filter(page_nrs: Optional[set[int]] = None) -> DoclingDocument
```

Create a new document based on the provided filter parameters.

`meth` get_visualization

```
get_visualization(show_label: bool = True, show_branch_numbering: bool = False, viz_mode:
Literal['reading_order', 'key_value'] = 'reading_order', show_cell_id: bool = False) -> dict[Optional[int],
Image]
```

Get visualization of the document as images by page.

Parameters:

- **`show_label`** (`bool`, default: `True`) – Show labels on elements (applies to all visualizers).
- **`show_branch_numbering`** (`bool`, default: `False`) – Show branch numbering (reading order visualizer only).
- **`visualizer`** (`str`) – Which visualizer to use. One of 'reading_order' (default), 'key_value'.
- **`show_cell_id`** (`bool`, default: `False`) – Show cell IDs (key value visualizer only).

Returns:

- `dict[Optional[int], PILImage.Image]` – Dictionary mapping page numbers to PIL images.

`meth` insert_code

```
insert_code(sibling: NodeItem, text: str, code_language: Optional[CodeLanguageLabel] = None, orig:
Optional[str] = None, caption: Optional[Union[TextItem, RefItem]] = None, prov: Optional[ProvenanceItem] =
None, content_layer: Optional[ContentLayer] = None, formatting: Optional[Formatting] = None, hyperlink:
Optional[Union[AnyUrl, Path]] = None, after: bool = True) -> CodeItem
```

Creates a new CodeItem item and inserts it into the document.

Parameters:

- **`sibling`** (`NodeItem`) – NodeItem:
- **`text`** (`str`) – str:
- **`code_language`** (`Optional[CodeLanguageLabel]`, default: `None`) – Optional[str]: (Default value = None)
- **`orig`** (`Optional[str]`, default: `None`) – Optional[str]: (Default value = None)
- **`caption`** (`Optional[Union[TextItem, RefItem]]`, default: `None`) – Optional[Union[TextItem, RefItem]]: (Default value = None)
- **`prov`** (`Optional[ProvenanceItem]`, default: `None`) – Optional[ProvenanceItem]: (Default value = None)
- **`content_layer`** (`Optional[ContentLayer]`, default: `None`) – Optional[ContentLayer]: (Default value = None)

- **formatting** (Optional[Formatting], default: None) – Optional[Formatting]: (Default value = None)
- **hyperlink** (Optional[Union[AnyUrl, Path]], default: None) – Optional[Union[AnyUrl, Path]]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- CodeItem – CodeItem: The newly created CodeItem item.

meth insert_document

```
insert_document(doc: DoclingDocument, sibling: NodeItem, after: bool = True) -> None
```

Inserts the content from the body of a DoclingDocument into this document at a specific position.

Parameters:

- **doc** (DoclingDocument) – DoclingDocument: The document whose content will be inserted
- **sibling** (NodeItem) – NodeItem: The NodeItem after/before which the new items will be inserted
- **after** (bool, default: True) – bool: If True, insert after the sibling; if False, insert before (Default value = True)

Returns:

- None – None

meth insert_form

```
insert_form(sibling: NodeItem, graph: GraphData, prov: Optional[ProvenanceItem] = None, after: bool = True)
-> FormItem
```

Creates a new FormItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **graph** (GraphData) – GraphData:
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- FormItem – FormItem: The newly created FormItem item.

meth insert_formula

```
insert_formula(sibling: NodeItem, text: str, orig: Optional[str] = None, prov: Optional[ProvenanceItem] =
None, content_layer: Optional[ContentLayer] = None, formatting: Optional[Formatting] = None, hyperlink:
Optional[Union[AnyUrl, Path]] = None, after: bool = True) -> FormulaItem
```

Creates a new FormulaItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:

- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)
- **formatting** (Optional[Formatting], default: None) – Optional[Formatting]: (Default value = None)
- **hyperlink** (Optional[Union[AnyUrl, Path]], default: None) – Optional[Union[AnyUrl, Path]]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- FormulaItem – FormulaItem: The newly created FormulaItem item.

meth insert_group

```
insert_group(sibling: NodeItem, label: Optional[GroupLabel] = None, name: Optional[str] = None,
content_layer: Optional[ContentLayer] = None, after: bool = True) -> GroupItem
```

Creates a new GroupItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **label** (Optional[GroupLabel], default: None) – Optional[GroupLabel]: (Default value = None)
- **name** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- GroupItem – GroupItem: The newly created GroupItem.

meth insert_heading

```
insert_heading(sibling: NodeItem, text: str, orig: Optional[str] = None, level: LevelNumber = 1, prov:
Optional[ProvenanceItem] = None, content_layer: Optional[ContentLayer] = None, formatting:
Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl, Path]] = None, after: bool = True) ->
SectionHeaderItem
```

Creates a new SectionHeaderItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **level** (LevelNumber, default: 1) – LevelNumber: (Default value = 1)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)

- **formatting** (Optional[Formatting], default: None) – Optional[Formatting]: (Default value = None)
- **hyperlink** (Optional[Union[AnyUrl, Path]], default: None) – Optional[Union[AnyUrl, Path]]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- SectionHeaderItem – SectionHeaderItem: The newly created SectionHeaderItem item.

meth **insert_inline_group**

```
insert_inline_group(sibling: NodeItem, name: Optional[str] = None, content_layer: Optional[ContentLayer] =
None, after: bool = True) -> InlineGroup
```

Creates a new InlineGroup item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **name** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- InlineGroup – InlineGroup: The newly created InlineGroup item.

meth **insert_item_after_sibling**

```
insert_item_after_sibling(*, new_item: NodeItem, sibling: NodeItem) -> None
```

Inserts an item, given its node_item instance, after other as a sibling.

meth **insert_item_before_sibling**

```
insert_item_before_sibling(*, new_item: NodeItem, sibling: NodeItem) -> None
```

Inserts an item, given its node_item instance, before other as a sibling.

meth **insert_key_values**

```
insert_key_values(sibling: NodeItem, graph: GraphData, prov: Optional ProvenanceItem = None, after: bool =
True) -> KeyValueItem
```

Creates a new KeyValueItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **graph** (GraphData) – GraphData:
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- `KeyValueItem` – KeyValueItem: The newly created KeyValueItem item.

`meth` **insert_list_group**

```
insert_list_group(sibling: NodeItem, name: Optional[str] = None, content_layer: Optional[ContentLayer] =
None, after: bool = True) -> ListGroup
```

Creates a new ListGroup item and inserts it into the document.

Parameters:

- **`sibling`** (`NodeItem`) – NodeItem:
- **`name`** (`Optional[str]`, default: `None`) – Optional[str]: (Default value = None)
- **`content_layer`** (`Optional[ContentLayer]`, default: `None`) – Optional[ContentLayer]: (Default value = None)
- **`after`** (`bool`, default: `True`) – bool: (Default value = True)

Returns:

- `ListGroup` – ListGroup: The newly created ListGroup item.

`meth` **insert_list_item**

```
insert_list_item(sibling: NodeItem, text: str, enumerated: bool = False, marker: Optional[str] = None, orig:
Optional[str] = None, prov: Optional[ProvenanceItem] = None, content_layer: Optional[ContentLayer] = None,
formatting: Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl, Path]] = None, after: bool = True)
-> ListItem
```

Creates a new ListItem item and inserts it into the document.

Parameters:

- **`sibling`** (`NodeItem`) – NodeItem:
- **`text`** (`str`) – str:
- **`enumerated`** (`bool`, default: `False`) – bool: (Default value = False)
- **`marker`** (`Optional[str]`, default: `None`) – Optional[str]: (Default value = None)
- **`orig`** (`Optional[str]`, default: `None`) – Optional[str]: (Default value = None)
- **`prov`** (`Optional[ProvenanceItem]`, default: `None`) – Optional[ProvenanceItem]: (Default value = None)
- **`content_layer`** (`Optional[ContentLayer]`, default: `None`) – Optional[ContentLayer]: (Default value = None)
- **`formatting`** (`Optional[Formatting]`, default: `None`) – Optional[Formatting]: (Default value = None)
- **`hyperlink`** (`Optional[Union[AnyUrl, Path]]`, default: `None`) – Optional[Union[AnyUrl, Path]]: (Default value = None)
- **`after`** (`bool`, default: `True`) – bool: (Default value = True)

Returns:

- `ListItem` – ListItem: The newly created ListItem item.

`meth` **insert_node_items**

```
insert_node_items(sibling: NodeItem, node_items: list[NodeItem], doc: DoclingDocument, after: bool = True) ->
None
```

Insert multiple NodeItems and their children at a specific position in the document.

Parameters:

- **sibling** (`NodeItem`) – NodeItem: The NodeItem after/before which the new items will be inserted
- **node_items** (`list[NodeItem]`) – list[NodeItem]: The NodeItems to be inserted
- **doc** (`DoclingDocument`) – DoclingDocument: The document to which the NodeItems and their children belong
- **after** (`bool`, default: `True`) – bool: If True, insert after the sibling; if False, insert before (Default value = True)

Returns:

- `None` – None

meth insert_picture

```
insert_picture(sibling: NodeItem, annotations: Optional[list[PictureDataType]] = None, image:
Optional[ImageRef] = None, caption: Optional[Union[TextItem, RefItem]] = None, prov: Optional[ProvenanceItem]
= None, content_layer: Optional[ContentLayer] = None, after: bool = True) -> PictureItem
```

Creates a new PictureItem item and inserts it into the document.

Parameters:

- **sibling** (`NodeItem`) – NodeItem:
- **annotations** (`Optional[list[PictureDataType]]`, default: `None`) – Optional[list[PictureDataType]]: (Default value = None)
- **image** (`Optional[ImageRef]`, default: `None`) – Optional[ImageRef]: (Default value = None)
- **caption** (`Optional[Union[TextItem, RefItem]]`, default: `None`) – Optional[Union[TextItem, RefItem]]: (Default value = None)
- **prov** (`Optional[ProvenanceItem]`, default: `None`) – Optional[ProvenanceItem]: (Default value = None)
- **content_layer** (`Optional[ContentLayer]`, default: `None`) – Optional[ContentLayer]: (Default value = None)
- **after** (`bool`, default: `True`) – bool: (Default value = True)

Returns:

- `PictureItem` – PictureItem: The newly created PictureItem item.

meth insert_table

```
insert_table(sibling: NodeItem, data: TableData, caption: Optional[Union[TextItem, RefItem]] = None, prov:
Optional[ProvenanceItem] = None, label: DocItemLabel = TABLE, content_layer: Optional[ContentLayer] = None,
annotations: Optional[list[TableAnnotationType]] = None, after: bool = True) -> TableItem
```

Creates a new TableItem item and inserts it into the document.

Parameters:

- **sibling** (`NodeItem`) – NodeItem:
- **data** (`TableData`) – TableData:

- **caption** (Optional[Union[TextItem, RefItem]], default: None) – Optional[Union[TextItem, RefItem]]: (Default value = None)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **label** (DocItemLabel, default: TABLE) – DocItemLabel: (Default value = DocItemLabel.TABLE)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)
- **annotations** (Optional[list[TableAnnotationType]], default: None) – Optional[list[TableAnnotationType]]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- TableItem – TableItem: The newly created TableItem item.

meth **insert_text**

```
insert_text(sibling: NodeItem, label: DocItemLabel, text: str, orig: Optional[str] = None, prov:
Optional[ProvenanceItem] = None, content_layer: Optional[ContentLayer] = None, formatting:
Optional[Formatting] = None, hyperlink: Optional[Union[AnyUrl, Path]] = None, after: bool = True) -> TextItem
```

Creates a new TextItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **label** (DocItemLabel) – DocItemLabel:
- **text** (str) – str:
- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)
- **formatting** (Optional[Formatting], default: None) – Optional[Formatting]: (Default value = None)
- **hyperlink** (Optional[Union[AnyUrl, Path]], default: None) – Optional[Union[AnyUrl, Path]]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- TextItem – TextItem: The newly created TextItem item.

meth **insert_title**

```
insert_title(sibling: NodeItem, text: str, orig: Optional[str] = None, prov: Optional[ProvenanceItem] = None,
content_layer: Optional[ContentLayer] = None, formatting: Optional[Formatting] = None, hyperlink:
Optional[Union[AnyUrl, Path]] = None, after: bool = True) -> TitleItem
```

Creates a new TitleItem item and inserts it into the document.

Parameters:

- **sibling** (NodeItem) – NodeItem:
- **text** (str) – str:

- **orig** (Optional[str], default: None) – Optional[str]: (Default value = None)
- **prov** (Optional[ProvenanceItem], default: None) – Optional[ProvenanceItem]: (Default value = None)
- **content_layer** (Optional[ContentLayer], default: None) – Optional[ContentLayer]: (Default value = None)
- **formatting** (Optional[Formatting], default: None) – Optional[Formatting]: (Default value = None)
- **hyperlink** (Optional[Union[AnyUrl, Path]], default: None) – Optional[Union[AnyUrl, Path]]: (Default value = None)
- **after** (bool, default: True) – bool: (Default value = True)

Returns:

- TitleItem – TitleItem: The newly created TitleItem item.

meth iterate_items

```
iterate_items(root: Optional[NodeItem] = None, with_groups: bool = False, traverse_pictures: bool = False, page_no: Optional[int] = None, included_content_layers: Optional[set[ContentLayer]] = None, _level: int = 0) -> Iterable[tuple[NodeItem, int]]
```

Iterate elements with level.

meth load_from_doctags

```
load_from_doctags(doctag_document: DocTagsDocument, document_name: str = 'Document') -> DoclingDocument
```

Load Docling document from lists of DocTags and Images.

meth load_from_json

```
load_from_json(filename: Union[str, Path]) -> DoclingDocument
```

load_from_json.

Parameters:

- **filename** (Union[str, Path]) – The filename to load a saved DoclingDocument from a .json.

Returns:

- DoclingDocument – The loaded DoclingDocument.

meth load_from_yaml

```
load_from_yaml(filename: Union[str, Path]) -> DoclingDocument
```

load_from_yaml.

Args: filename: The filename to load a YAML-serialized DoclingDocument from.

Returns: DoclingDocument: the loaded DoclingDocument

meth num_pages

```
num_pages()
```

num_pages.

meth **print_element_tree**

```
print_element_tree()
```

Print_element_tree.

meth **replace_item**

```
replace_item(*, new_item: NodeItem, old_item: NodeItem) -> None
```

Replace item with new item.

meth **save_as_doctags**

```
save_as_doctags(filename: Union[str, Path], delim: str = '', from_element: int = 0, to_element: int = maxsize, labels: Optional[set[DocItemLabel]] = None, xsize: int = 500, ysize: int = 500, add_location: bool = True, add_content: bool = True, add_page_index: bool = True, add_table_cell_location: bool = False, add_table_cell_text: bool = True, minified: bool = False)
```

Save the document content to DocTags format.

meth **save_as_document_tokens**

```
save_as_document_tokens(*args, **kwargs)
```

Save the document content to a DocumentToken format.

meth **save_as_html**

```
save_as_html(filename: Union[str, Path], artifacts_dir: Optional[Path] = None, from_element: int = 0, to_element: int = maxsize, labels: Optional[set[DocItemLabel]] = None, image_mode: ImageRefMode = PLACEHOLDER, formula_to_mathml: bool = True, page_no: Optional[int] = None, html_lang: str = 'en', html_head: str = 'null', included_content_layers: Optional[set[ContentLayer]] = None, split_page_view: bool = False, include_annotations: bool = True)
```

Save to HTML.

meth **save_as_json**

```
save_as_json(filename: Union[str, Path], artifacts_dir: Optional[Path] = None, image_mode: ImageRefMode = EMBEDDED, indent: int = 2, coord_precision: Optional[int] = None, confid_precision: Optional[int] = None)
```

Save as json.

meth **save_as_markdown**

```
save_as_markdown(filename: Union[str, Path], artifacts_dir: Optional[Path] = None, delim: str = '\n\n', from_element: int = 0, to_element: int = maxsize, labels: Optional[set[DocItemLabel]] = None, strict_text: bool = False, escape_html: bool = True, escaping_underscores: bool = True, image_placeholder: str = '<!-- image -->', image_mode: ImageRefMode = PLACEHOLDER, indent: int = 4, text_width: int = -1, page_no: Optional[int] = None, included_content_layers: Optional[set[ContentLayer]] = None, page_break_placeholder: Optional[str] = None, include_annotations: bool = True, compact_tables: bool = False, *, mark_meta: bool = False, use_legacy_annotations: Optional[bool] = None)
```

Save to markdown.

`meth` save_as_vtt

```
save_as_vtt(filename: str | Path, included_content_layers: set[ContentLayer] | None = None,
omit_hours_if_zero: bool = False, omit_voice_end: bool = True) -> None
```

Saves the Docling document to a file in WebVTT format.

Args: filename: The path to the WebVTT file. included_content_layers: The content layers to serializes. If ommitted, the `DEFAULT_CONTENT_LAYERS` will be serialized. omit_hours_if_zero: If True, omit hours when they are 0 in the timings. omit_voice_end: If True and cue blocks have a WebVTT cue voice span as the only component, omit the voice end tag for brevity.

`meth` save_as_yaml

```
save_as_yaml(filename: Union[str, Path], artifacts_dir: Optional[Path] = None, image_mode: ImageRefMode =
EMBEDDED, default_flow_style: bool = False, coord_precision: Optional[int] = None, confid_precision:
Optional[int] = None)
```

Save as yaml.

`meth` transform_to_content_layer

```
transform_to_content_layer(data: Any) -> Any
```

transform_to_content_layer.

`meth` validate_document

```
validate_document() -> Self
```

validate_document.

`meth` validate_misplaced_list_items

```
validate_misplaced_list_items() -> Self
```

validate_misplaced_list_items.

`meth` validate_tree

```
validate_tree(root: NodeItem) -> bool
```

validate_tree.

`class` DocumentOrigin

Bases: `BaseModel`

FileSource.

Methods:

- `parse_hex_string` – parse_hex_string.

- `validate_mimetype` – validate_mimetype.

Attributes:

- `binary_hash` (`Uint64`) –
- `filename` (`str`) –
- `mimetype` (`str`) –
- `uri` (`Optional[AnyUrl]`) –

`attr` **binary_hash**

```
binary_hash: Uint64
```

`attr` **filename**

```
filename: str
```

`attr` **mimetype**

```
mimetype: str
```

`attr` **uri**

```
uri: Optional[AnyUrl] = None
```

`meth` **parse_hex_string**

```
parse_hex_string(value)
```

parse_hex_string.

`meth` **validate_mimetype**

```
validate_mimetype(v)
```

validate_mimetype.

`class` DocItem

Bases: `NodeItem`

Base type for any element that carries content, can be a leaf node.

Methods:

- `get_annotations` – Get the annotations of this DocItem.
- `get_image` – Returns the image of this DocItem.
- `get_location_tokens` – Get the location string for the BaseCell.
- `get_ref` – get_ref.

Attributes:

- **children** (list[RefItem]) –
- **comments** (list[FineRef]) –
- **content_layer** (ContentLayer) –
- **label** (DocItemLabel) –
- **meta** (Optional[BaseMeta]) –
- **model_config** –
- **parent** (Optional[RefItem]) –
- **prov** (list[ProvenanceItem]) –
- **self_ref** (str) –
- **source** (Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]) –

attr children

```
children: list[RefItem] = []
```

attr comments

```
comments: list[FineRef] = []
```

attr content_layer

```
content_layer: ContentLayer = BODY
```

attr label

```
label: DocItemLabel
```

attr meta

```
meta: Optional[BaseMeta] = None
```

attr model_config

```
model_config = ConfigDict(extra='forbid')
```

attr parent

```
parent: Optional[RefItem] = None
```

attr prov

```
prov: list[ProvenanceItem] = []
```

attr self_ref

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

attr **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

meth **get_annotations**

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this DocItem.

meth **get_image**

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image of this DocItem.

The function returns None if this DocItem has no valid provenance or if a valid image of the page containing this DocItem is not available in doc.

meth **get_location_tokens**

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500, self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

meth **get_ref**

```
get_ref() -> RefItem
```

get_ref.

class DocItemLabel

Bases: `str`, `Enum`

DocItemLabel.

Methods:

- `get_color` – Return the RGB color associated with a given label.

Attributes:

- `CAPTION` –
- `CHART` –
- `CHECKBOX_SELECTED` –
- `CHECKBOX_UNSELECTED` –

- `CODE` –
- `DOCUMENT_INDEX` –
- `EMPTY_VALUE` –
- `FOOTNOTE` –
- `FORM` –
- `FORMULA` –
- `GRADING_SCALE` –
- `HANDWRITTEN_TEXT` –
- `KEY_VALUE_REGION` –
- `LIST_ITEM` –
- `PAGE_FOOTER` –
- `PAGE_HEADER` –
- `PARAGRAPH` –
- `PICTURE` –
- `REFERENCE` –
- `SECTION_HEADER` –
- `TABLE` –
- `TEXT` –
- `TITLE` –

`attr` **CAPTION**

```
CAPTION = 'caption'
```

`attr` **CHART**

```
CHART = 'chart'
```

`attr` **CHECKBOX_SELECTED**

```
CHECKBOX_SELECTED = 'checkbox_selected'
```

`attr` **CHECKBOX_UNSELECTED**

```
CHECKBOX_UNSELECTED = 'checkbox_unselected'
```

`attr` **CODE**

```
CODE = 'code'
```

`attr` **DOCUMENT_INDEX**

```
DOCUMENT_INDEX = 'document_index'
```

attr **EMPTY_VALUE**

```
EMPTY_VALUE = 'empty_value'
```

attr **FOOTNOTE**

```
FOOTNOTE = 'footnote'
```

attr **FORM**

```
FORM = 'form'
```

attr **FORMULA**

```
FORMULA = 'formula'
```

attr **GRADING_SCALE**

```
GRADING_SCALE = 'grading_scale'
```

attr **HANDWRITTEN_TEXT**

```
HANDWRITTEN_TEXT = 'handwritten_text'
```

attr **KEY_VALUE_REGION**

```
KEY_VALUE_REGION = 'key_value_region'
```

attr **LIST_ITEM**

```
LIST_ITEM = 'list_item'
```

attr **PAGE_FOOTER**

```
PAGE_FOOTER = 'page_footer'
```

attr **PAGE_HEADER**

```
PAGE_HEADER = 'page_header'
```

attr **PARAGRAPH**

```
PARAGRAPH = 'paragraph'
```

attr **PICTURE**

```
PICTURE = 'picture'
```

attr **REFERENCE**

```
REFERENCE = 'reference'
```

attr SECTION_HEADER

```
SECTION_HEADER = 'section_header'
```

attr TABLE

```
TABLE = 'table'
```

attr TEXT

```
TEXT = 'text'
```

attr TITLE

```
TITLE = 'title'
```

meth get_color

```
get_color(label: DocItemLabel) -> tuple[int, int, int]
```

Return the RGB color associated with a given label.

class ProvenanceItem

Bases: `BaseModel`

Provenance information for elements extracted from a textual document.

A `ProvenanceItem` object acts as a lightweight pointer back into the original document for an extracted element. It applies to documents with an explicity or implicit layout, such as PDF, HTML, docx, or pptx.

Attributes:

- **bbox** (`Annotated[BoundingBox, Field(description='Bounding box')]`) –
- **charspan** (`Annotated[tuple[int, int], Field(description='Character span (0-indexed)')]`) –
- **page_no** (`Annotated[int, Field(description='Page number')]`) –

attr bbox

```
bbox: Annotated[BoundingBox, Field(description='Bounding box')]
```

attr charspan

```
charspan: Annotated[tuple[int, int], Field(description='Character span (0-indexed)')]
```

attr page_no

```
page_no: Annotated[int, Field(description='Page number')]
```

class GroupItem

Bases: NodeItem

GroupItem.

Methods:

- **get_ref** – get_ref.

Attributes:

- **children** (list[RefItem]) –
- **content_layer** (ContentLayer) –
- **label** (GroupLabel) –
- **meta** (Optional[BaseMeta]) –
- **model_config** –
- **name** (str) –
- **parent** (Optional[RefItem]) –
- **self_ref** (str) –

attr children

```
children: list[RefItem] = []
```

attr content_layer

```
content_layer: ContentLayer = BODY
```

attr label

```
label: GroupLabel = UNSPECIFIED
```

attr meta

```
meta: Optional[BaseMeta] = None
```

attr model_config

```
model_config = ConfigDict(extra='forbid')
```

attr name

```
name: str = 'group'
```

attr parent

```
parent: Optional[RefItem] = None
```

attr self_ref

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

meth **get_ref**

```
get_ref() -> RefItem
```

get_ref.

class GroupLabel

Bases: `str`, `Enum`

GroupLabel.

Attributes:

- `CHAPTER` –
- `COMMENT_SECTION` –
- `FORM_AREA` –
- `INLINE` –
- `KEY_VALUE_AREA` –
- `LIST` –
- `ORDERED_LIST` –
- `PICTURE_AREA` –
- `SECTION` –
- `SHEET` –
- `SLIDE` –
- `UNSPECIFIED` –

attr **CHAPTER**

```
CHAPTER = 'chapter'
```

attr **COMMENT_SECTION**

```
COMMENT_SECTION = 'comment_section'
```

attr **FORM_AREA**

```
FORM_AREA = 'form_area'
```

attr **INLINE**

```
INLINE = 'inline'
```

attr **KEY_VALUE_AREA**

```
KEY_VALUE_AREA = 'key_value_area'
```

attr **LIST**

```
LIST = 'list'
```

attr **ORDERED_LIST**

```
ORDERED_LIST = 'ordered_list'
```

attr **PICTURE_AREA**

```
PICTURE_AREA = 'picture_area'
```

attr **SECTION**

```
SECTION = 'section'
```

attr **SHEET**

```
SHEET = 'sheet'
```

attr **SLIDE**

```
SLIDE = 'slide'
```

attr **UNSPECIFIED**

```
UNSPECIFIED = 'unspecified'
```

class NodeItem

Bases: `BaseModel`

NodeItem.

Methods:

- **get_ref** – get_ref.

Attributes:

- **children** (list[RefItem]) –
- **content_layer** (ContentLayer) –
- **meta** (Optional[BaseMeta]) –
- **model_config** –
- **parent** (Optional[RefItem]) –
- **self_ref** (str) –

attr **children**

```
children: list[RefItem] = []
```

attr **content_layer**

```
content_layer: ContentLayer = BODY
```

attr **meta**

```
meta: Optional[BaseMeta] = None
```

attr **model_config**

```
model_config = ConfigDict(extra='forbid')
```

attr **parent**

```
parent: Optional[RefItem] = None
```

attr **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

meth **get_ref**

```
get_ref() -> RefItem
```

get_ref.

class PageItem

Bases: BaseModel

PageItem.

Attributes:

- **image** (Optional[ImageRef]) –
- **page_no** (int) –
- **size** (Size) –

attr **image**

```
image: Optional[ImageRef] = None
```

attr **page_no**

```
page_no: int
```

attr **size**

```
size: Size
```

class FloatingItem

Bases: DocItem

FloatingItem.

Methods:

- **caption_text** – Computes the caption as a single text.
- **get_annotations** – Get the annotations of this DocItem.
- **get_image** – Returns the image corresponding to this FloatingItem.
- **get_location_tokens** – Get the location string for the BaseCell.
- **get_ref** – get_ref.

Attributes:

- **captions** (list[RefItem]) –
- **children** (list[RefItem]) –
- **comments** (list[FineRef]) –
- **content_layer** (ContentLayer) –
- **footnotes** (list[RefItem]) –
- **image** (Optional[ImageRef]) –
- **label** (DocItemLabel) –
- **meta** (Optional[FloatingMeta]) –
- **model_config** –
- **parent** (Optional[RefItem]) –
- **prov** (list[ProvenanceItem]) –
- **references** (list[RefItem]) –
- **self_ref** (str) –
- **source** (Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]) –

attr captions

```
captions: list[RefItem] = []
```

attr children

```
children: list[RefItem] = []
```

attr comments

```
comments: list[FineRef] = []
```

attr **content_layer**

```
content_layer: ContentLayer = BODY
```

attr **footnotes**

```
footnotes: list[RefItem] = []
```

attr **image**

```
image: Optional[ImageRef] = None
```

attr **label**

```
label: DocItemLabel
```

attr **meta**

```
meta: Optional[FloatingMeta] = None
```

attr **model_config**

```
model_config = ConfigDict(extra='forbid')
```

attr **parent**

```
parent: Optional[RefItem] = None
```

attr **prov**

```
prov: list[ProvenanceItem] = []
```

attr **references**

```
references: list[RefItem] = []
```

attr **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

attr **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

meth **caption_text**

```
caption_text(doc: DoclingDocument) -> str
```

Computes the caption as a single text.

meth **get_annotations**

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this DocItem.

meth **get_image**

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image corresponding to this FloatingItem.

This function returns the PIL image from self.image if one is available. Otherwise, it uses DocItem.get_image to get an image of this FloatingItem.

In particular, when self.image is None, the function returns None if this FloatingItem has no valid provenance or the doc does not contain a valid image for the required page.

meth **get_location_tokens**

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500,
self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

meth **get_ref**

```
get_ref() -> RefItem
```

get_ref.

class TextItem

Bases: `DocItem`

TextItem.

Methods:

- **export_to_doctags** – Export text element to document tokens format.
- **export_to_document_tokens** – Export to DocTags format.
- **get_annotations** – Get the annotations of this DocItem.
- **get_image** – Returns the image of this DocItem.
- **get_location_tokens** – Get the location string for the BaseCell.
- **get_ref** – get_ref.

Attributes:

- **children** (list[RefItem]) –
- **comments** (list[FineRef]) –
- **content_layer** (ContentLayer) –

- **formatting** (Optional[Formatting]) –
- **hyperlink** (Optional[Union[AnyUrl, Path]]) –
- **label** (Literal[CAPTION, CHECKBOX_SELECTED, CHECKBOX_UNSELECTED, FOOTNOTE, PAGE_FOOTER, PAGE_HEADER, PARAGRAPH, REFERENCE, TEXT, EMPTY_VALUE]) –
- **meta** (Optional[BaseMeta]) –
- **model_config** –
- **orig** (str) –
- **parent** (Optional[RefItem]) –
- **prov** (list[ProvenanceItem]) –
- **self_ref** (str) –
- **source** (Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]) –
- **text** (str) –

attr **children**

```
children: list[RefItem] = []
```

attr **comments**

```
comments: list[FineRef] = []
```

attr **content_layer**

```
content_layer: ContentLayer = BODY
```

attr **formatting**

```
formatting: Optional[Formatting] = None
```

attr **hyperlink**

```
hyperlink: Optional[Union[AnyUrl, Path]] = Field(union_mode='left_to_right', default None)
```

attr **label**

```
label: Literal[CAPTION, CHECKBOX_SELECTED, CHECKBOX_UNSELECTED, FOOTNOTE, PAGE_FOOTER, PAGE_HEADER, PARAGRAPH, REFERENCE, TEXT, EMPTY_VALUE]
```

attr **meta**

```
meta: Optional[BaseMeta] = None
```

attr **model_config**

```
model_config = ConfigDict(extra='forbid')
```

attr **orig**

```
orig: str
```

attr **parent**

```
parent: Optional[RefItem] = None
```

attr **prov**

```
prov: list[ProvenanceItem] = []
```

attr **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

attr **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

attr **text**

```
text: str
```

meth **export_to_doctags**

```
export_to_doctags(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500, add_location: bool = True, add_content: bool = True)
```

Export text element to document tokens format.

Parameters:

- **doc** (`DoclingDocument`) – "DoclingDocument":
- **new_line** (`str`, default: `''`) – str (Default value = "") Deprecated
- **xsize** (`int`, default: `500`) – int: (Default value = 500)
- **ysize** (`int`, default: `500`) – int: (Default value = 500)
- **add_location** (`bool`, default: `True`) – bool: (Default value = True)
- **add_content** (`bool`, default: `True`) – bool: (Default value = True)

meth **export_to_document_tokens**

```
export_to_document_tokens(*args, **kwargs)
```

Export to DocTags format.

meth **get_annotations**

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this DocItem.

meth **get_image**

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image of this DocItem.

The function returns None if this DocItem has no valid provenance or if a valid image of the page containing this DocItem is not available in doc.

meth **get_location_tokens**

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500,
self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

meth **get_ref**

```
get_ref() -> RefItem
```

get_ref.

class TableItem

Bases: `FloatingItem`

TableItem.

Methods:

- **`add_annotation`** – Add an annotation to the table.
- **`caption_text`** – Computes the caption as a single text.
- **`export_to_dataframe`** – Export the table as a Pandas DataFrame.
- **`export_to_doctags`** – Export table to document tokens format.
- **`export_to_document_tokens`** – Export to DocTags format.
- **`export_to_html`** – Export the table as html.
- **`export_to_markdown`** – Export the table as markdown.
- **`export_to_otsl`** – Export the table as OTSL.
- **`get_annotations`** – Get the annotations of this TableItem.
- **`get_image`** – Returns the image corresponding to this FloatingItem.
- **`get_location_tokens`** – Get the location string for the BaseCell.
- **`get_ref`** – get_ref.

Attributes:

- **`annotations`** (`Annotated[list[TableAnnotationType], deprecated('Field `annotations` is deprecated; use `meta` instead.')]`) –

- **captions** (list[RefItem]) –
- **children** (list[RefItem]) –
- **comments** (list[FineRef]) –
- **content_layer** (ContentLayer) –
- **data** (TableData) –
- **footnotes** (list[RefItem]) –
- **image** (Optional[ImageRef]) –
- **label** (Literal[DOCUMENT_INDEX, TABLE]) –
- **meta** (Optional[FloatingMeta]) –
- **model_config** –
- **parent** (Optional[RefItem]) –
- **prov** (list[ProvenanceItem]) –
- **references** (list[RefItem]) –
- **self_ref** (str) –
- **source** (Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]) –

attr annotations

```
annotations: Annotated[list[TableAnnotationType], deprecated('Field `annotations` is deprecated; use `meta` instead.')] = []
```

attr captions

```
captions: list[RefItem] = []
```

attr children

```
children: list[RefItem] = []
```

attr comments

```
comments: list[FineRef] = []
```

attr content_layer

```
content_layer: ContentLayer = BODY
```

attr data

```
data: TableData
```

attr footnotes

```
footnotes: list[RefItem] = []
```

attr **image**

```
image: Optional[ImageRef] = None
```

attr **label**

```
label: Literal[DOCUMENT_INDEX, TABLE] = TABLE
```

attr **meta**

```
meta: Optional[FloatingMeta] = None
```

attr **model_config**

```
model_config = ConfigDict(extra='forbid')
```

attr **parent**

```
parent: Optional[RefItem] = None
```

attr **prov**

```
prov: list[ProvenanceItem] = []
```

attr **references**

```
references: list[RefItem] = []
```

attr **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

attr **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

meth **add_annotation**

```
add_annotation(annotation: TableAnnotationType) -> None
```

Add an annotation to the table.

meth **caption_text**

```
caption_text(doc: DoclingDocument) -> str
```

Computes the caption as a single text.

meth **export_to_dataframe**

```
export_to_dataframe(doc: Optional[DoclingDocument] = None) -> DataFrame
```

Export the table as a Pandas DataFrame.

meth **export_to_doctags**

```
export_to_doctags(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500, add_location:
bool = True, add_cell_location: bool = True, add_cell_text: bool = True, add_caption: bool = True)
```

Export table to document tokens format.

Parameters:

- **doc** (DoclingDocument) – "DoclingDocument":
- **new_line** (str, default: '') – str (Default value = "") Deprecated
- **xsize** (int, default: 500) – int: (Default value = 500)
- **ysize** (int, default: 500) – int: (Default value = 500)
- **add_location** (bool, default: True) – bool: (Default value = True)
- **add_cell_location** (bool, default: True) – bool: (Default value = True)
- **add_cell_text** (bool, default: True) – bool: (Default value = True)
- **add_caption** (bool, default: True) – bool: (Default value = True)

meth **export_to_document_tokens**

```
export_to_document_tokens(*args, **kwargs)
```

Export to DocTags format.

meth **export_to_html**

```
export_to_html(doc: Optional[DoclingDocument] = None, add_caption: bool = True) -> str
```

Export the table as html.

meth **export_to_markdown**

```
export_to_markdown(doc: Optional[DoclingDocument] = None) -> str
```

Export the table as markdown.

meth **export_to_otsl**

```
export_to_otsl(doc: DoclingDocument, add_cell_location: bool = True, add_cell_text: bool = True, xsize: int =
500, ysize: int = 500, self_closing: bool = False, **kwargs: Any) -> str
```

Export the table as OTSL.

meth **get_annotations**

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this TableItem.

`meth` get_image

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image corresponding to this FloatingItem.

This function returns the PIL image from self.image if one is available. Otherwise, it uses DocItem.get_image to get an image of this FloatingItem.

In particular, when self.image is None, the function returns None if this FloatingItem has no valid provenance or the doc does not contain a valid image for the required page.

`meth` get_location_tokens

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500,
self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

`meth` get_ref

```
get_ref() -> RefItem
```

get_ref.

`class` TableCell

Bases: `BaseModel`

TableCell.

Methods:

- `from_dict_format` – from_dict_format.

Attributes:

- `bbox` (`Optional[BoundingBox]`) –
- `col_span` (`int`) –
- `column_header` (`bool`) –
- `end_col_offset_idx` (`int`) –
- `end_row_offset_idx` (`int`) –
- `fillable` (`bool`) –
- `row_header` (`bool`) –
- `row_section` (`bool`) –
- `row_span` (`int`) –
- `start_col_offset_idx` (`int`) –

- `start_row_offset_idx` (`int`) –
- `text` (`str`) –

attr **bbox**

```
bbox: Optional[BoundingBox] = None
```

attr **col_span**

```
col_span: int = 1
```

attr **column_header**

```
column_header: bool = False
```

attr **end_col_offset_idx**

```
end_col_offset_idx: int
```

attr **end_row_offset_idx**

```
end_row_offset_idx: int
```

attr **fillable**

```
fillable: bool = False
```

attr **row_header**

```
row_header: bool = False
```

attr **row_section**

```
row_section: bool = False
```

attr **row_span**

```
row_span: int = 1
```

attr **start_col_offset_idx**

```
start_col_offset_idx: int
```

attr **start_row_offset_idx**

```
start_row_offset_idx: int
```

attr **text**

```
text: str
```

meth **from_dict_format**

```
from_dict_format(data: Any) -> Any
```

from_dict_format.

class TableData

Bases: `BaseModel`

BaseTableData.

Methods:

- `add_row` – Add a new row to the table from a list of strings.
- `add_rows` – Add multiple new rows to the table from a list of lists of strings.
- `from_regions` – Converts regions: rows, columns, merged cells into table_data structure.
- `get_column_bounding_boxes` – Get the bounding box for each column in the table.
- `get_row_bounding_boxes` – Get the bounding box for each row in the table.
- `insert_row` – Insert a new row from a list of strings before/after a specific index in the table.
- `insert_rows` – Insert multiple new rows from a list of lists of strings before/after a specific index in the table.
- `pop_row` – Remove and return the last row from the table.
- `remove_row` – Remove a row from the table by its index.
- `remove_rows` – Remove rows from the table by their indices.

Attributes:

- `grid` (`list[list[TableCell]]`) – grid.
- `num_cols` (`int`) –
- `num_rows` (`int`) –
- `table_cells` (`list[AnyTableCell]`) –

attr **grid**

```
grid: list[list[TableCell]]
```

grid.

attr **num_cols**

```
num_cols: int = 0
```

attr **num_rows**

```
num_rows: int = 0
```

attr **table_cells**

```
table_cells: list[AnyTableCell] = []
```

meth **add_row**

```
add_row(row: list[str]) -> None
```

Add a new row to the table from a list of strings.

Parameters:

- **row** (`list[str]`) – list[str]: A list of strings representing the content of the new row.

Returns:

- `None` – None

meth **add_rows**

```
add_rows(rows: list[list[str]]) -> None
```

Add multiple new rows to the table from a list of lists of strings.

Parameters:

- **rows** (`list[list[str]]`) – list[list[str]]: A list of lists, where each inner list represents the content of a new row.

Returns:

- `None` – None

meth **from_regions**

```
from_regions(table_bbox: BoundingBox, rows: list[BoundingBox], cols: list[BoundingBox], merges:
list[BoundingBox], row_headers: list[BoundingBox] = [], col_headers: list[BoundingBox] = [], row_sections:
list[BoundingBox] = []) -> Self
```

Converts regions: rows, columns, merged cells into table_data structure.

Adds semantics for regions of row_headers, col_headers, row_section

meth **get_column_bounding_boxes**

```
get_column_bounding_boxes(*, minimal: bool = True) -> dict[int, BoundingBox]
```

Get the bounding box for each column in the table.

Args: minimal: If True (default), returns the minimal bounding box for each column based on its cells. If False, all columns will have uniform vertical extent (same y0/y1 values) spanning the full table height.

Returns: dict[int, BoundingBox]: A dictionary mapping column indices to their bounding boxes. Only columns with cells that have bounding boxes are included.

meth **get_row_bounding_boxes**

```
get_row_bounding_boxes(*, minimal: bool = True) -> dict[int, BoundingBox]
```

Get the bounding box for each row in the table.

Args: minimal: If True (default), returns the minimal bounding box for each row based on its cells. If False, all rows will have uniform horizontal extent (same x0/x1 values) spanning the full table width.

Returns: dict[int, BoundingBox]: A dictionary mapping row indices to their bounding boxes. Only rows with cells that have bounding boxes are included.

`meth` insert_row

```
insert_row(row_index: int, row: list[str], after: bool = False) -> None
```

Insert a new row from a list of strings before/after a specific index in the table.

Parameters:

- **`row_index`** (`int`) – int: The index at which to insert the new row. (Starting from 0)
- **`row`** (`list[str]`) – list[str]: A list of strings representing the content of the new row.
- **`after`** (`bool`, default: `False`) – bool: If True, insert the row after the specified index, otherwise before it. (Default is False)

Returns:

- `None` – None

`meth` insert_rows

```
insert_rows(row_index: int, rows: list[list[str]], after: bool = False) -> None
```

Insert multiple new rows from a list of lists of strings before/after a specific index in the table.

Parameters:

- **`row_index`** (`int`) – int: The index at which to insert the new rows. (Starting from 0)
- **`rows`** (`list[list[str]]`) – list[list[str]]: A list of lists, where each inner list represents the content of a new row.
- **`after`** (`bool`, default: `False`) – bool: If True, insert the rows after the specified index, otherwise before it. (Default is False)

Returns:

- `None` – None

`meth` pop_row

```
pop_row(doc: Optional[DoclingDocument] = None) -> list[TableCell]
```

Remove and return the last row from the table.

Returns:

- `list[TableCell]` – list[TableCell]: A list of TableCell objects representing the popped row.

`meth` remove_row

```
remove_row(row_index: int, doc: Optional[DoclingDocument] = None) -> list[TableCell]
```

Remove a row from the table by its index.

Parameters:

- **row_index** (int) – int: The index of the row to remove. (Starting from 0)

Returns:

- list[TableCell] – list[TableCell]: A list of TableCell objects representing the removed row.

meth **remove_rows**

```
remove_rows(indices: list[int], doc: Optional[DoclingDocument] = None) -> list[list[TableCell]]
```

Remove rows from the table by their indices.

Parameters:

- **indices** (list[int]) – list[int]: A list of indices of the rows to remove. (Starting from 0)

Returns:

- list[list[TableCell]] – list[list[TableCell]]: A list representation of the removed rows as lists of TableCell objects.

class TableCellLabel

Bases: str, Enum

TableCellLabel.

Methods:

- **get_color** – Return the RGB color associated with a given label.

Attributes:

- **BODY** –
- **COLUMN_HEADER** –
- **ROW_HEADER** –
- **ROW_SECTION** –

attr **BODY**

```
BODY = 'body'
```

attr **COLUMN_HEADER**

```
COLUMN_HEADER = 'col_header'
```

attr **ROW_HEADER**

```
ROW_HEADER = 'row_header'
```

attr **ROW_SECTION**

```
ROW_SECTION = 'row_section'
```

meth **get_color**

```
get_color(label: TableCellLabel) -> tuple[int, int, int]
```

Return the RGB color associated with a given label.

class KeyValueItem

Bases: `FloatingItem`

KeyValueItem.

Methods:

- **caption_text** – Computes the caption as a single text.
- **export_to_document_tokens** – Export key value item to document tokens format.
- **get_annotations** – Get the annotations of this DocItem.
- **get_image** – Returns the image corresponding to this FloatingItem.
- **get_location_tokens** – Get the location string for the BaseCell.
- **get_ref** – get_ref.

Attributes:

- **captions** (list[RefItem]) –
- **children** (list[RefItem]) –
- **comments** (list[FineRef]) –
- **content_layer** (ContentLayer) –
- **footnotes** (list[RefItem]) –
- **graph** (GraphData) –
- **image** (Optional[ImageRef]) –
- **label** (Literal[KEY_VALUE_REGION]) –
- **meta** (Optional[FloatingMeta]) –
- **model_config** –
- **parent** (Optional[RefItem]) –
- **prov** (list[ProvenanceItem]) –
- **references** (list[RefItem]) –
- **self_ref** (str) –
- **source** (Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]) –

attr **captions**

```
captions: list[RefItem] = []
```

attr children

```
children: list[RefItem] = []
```

attr comments

```
comments: list[FineRef] = []
```

attr content_layer

```
content_layer: ContentLayer = BODY
```

attr footnotes

```
footnotes: list[RefItem] = []
```

attr graph

```
graph: GraphData
```

attr image

```
image: Optional[ImageRef] = None
```

attr label

```
label: Literal[KEY_VALUE_REGION] = KEY_VALUE_REGION
```

attr meta

```
meta: Optional[FloatingMeta] = None
```

attr model_config

```
model_config = ConfigDict(extra='forbid')
```

attr parent

```
parent: Optional[RefItem] = None
```

attr prov

```
prov: list[ProvenanceItem] = []
```

attr references

```
references: list[RefItem] = []
```

`attr` **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

`attr` **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

`meth` **caption_text**

```
caption_text(doc: DoclingDocument) -> str
```

Computes the caption as a single text.

`meth` **export_to_document_tokens**

```
export_to_document_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500, add_location: bool = True, add_content: bool = True)
```

Export key value item to document tokens format.

Parameters:

- **doc** (DoclingDocument) – "DoclingDocument":
- **new_line** (str, default: '') – str (Default value = "") Deprecated
- **xsize** (int, default: 500) – int: (Default value = 500)
- **ysize** (int, default: 500) – int: (Default value = 500)
- **add_location** (bool, default: True) – bool: (Default value = True)
- **add_content** (bool, default: True) – bool: (Default value = True)

`meth` **get_annotations**

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this DocItem.

`meth` **get_image**

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image corresponding to this FloatingItem.

This function returns the PIL image from self.image if one is available. Otherwise, it uses DocItem.get_image to get an image of this FloatingItem.

In particular, when self.image is None, the function returns None if this FloatingItem has no valid provenance or the doc does not contain a valid image for the required page.

`meth` **get_location_tokens**

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500,
self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

meth **get_ref**

```
get_ref() -> RefItem
```

get_ref.

class SectionHeaderItem

Bases: `TextItem`

SectionItem.

Methods:

- **export_to_doctags** – Export text element to document tokens format.
- **export_to_document_tokens** – Export to DocTags format.
- **get_annotations** – Get the annotations of this DocItem.
- **get_image** – Returns the image of this DocItem.
- **get_location_tokens** – Get the location string for the BaseCell.
- **get_ref** – get_ref.

Attributes:

- **children** (`list[RefItem]`) –
- **comments** (`list[FineRef]`) –
- **content_layer** (`ContentLayer`) –
- **formatting** (`Optional[Formatting]`) –
- **hyperlink** (`Optional[Union[AnyUrl, Path]]`) –
- **label** (`Literal[SECTION_HEADER]`) –
- **level** (`LevelNumber`) –
- **meta** (`Optional[BaseMeta]`) –
- **model_config** –
- **orig** (`str`) –
- **parent** (`Optional[RefItem]`) –
- **prov** (`list[ProvenanceItem]`) –
- **self_ref** (`str`) –
- **source** (`Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]`) –
- **text** (`str`) –

attr **children**

```
children: list[RefItem] = []
```

attr **comments**

```
comments: list[FineRef] = []
```

attr **content_layer**

```
content_layer: ContentLayer = BODY
```

attr **formatting**

```
formatting: Optional[Formatting] = None
```

attr **hyperlink**

```
hyperlink: Optional[Union[AnyUrl, Path]] = Field(union_mode='left_to_right', default None)
```

attr **label**

```
label: Literal[SECTION_HEADER] = SECTION_HEADER
```

attr **level**

```
level: LevelNumber = 1
```

attr **meta**

```
meta: Optional[BaseMeta] = None
```

attr **model_config**

```
model_config = ConfigDict(extra='forbid')
```

attr **orig**

```
orig: str
```

attr **parent**

```
parent: Optional[RefItem] = None
```

attr **prov**

```
prov: list[ProvenanceItem] = []
```

attr **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

attr **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

attr **text**

```
text: str
```

meth **export_to_doctags**

```
export_to_doctags(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500, add_location: bool = True, add_content: bool = True)
```

Export text element to document tokens format.

Parameters:

- **doc** (DoclingDocument) – "DoclingDocument":
- **new_line** (str, default: '') – str (Default value = "") Deprecated
- **xsize** (int, default: 500) – int: (Default value = 500)
- **ysize** (int, default: 500) – int: (Default value = 500)
- **add_location** (bool, default: True) – bool: (Default value = True)
- **add_content** (bool, default: True) – bool: (Default value = True)

meth **export_to_document_tokens**

```
export_to_document_tokens(*args, **kwargs)
```

Export to DocTags format.

meth **get_annotations**

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this DocItem.

meth **get_image**

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image of this DocItem.

The function returns None if this DocItem has no valid provenance or if a valid image of the page containing this DocItem is not available in doc.

meth **get_location_tokens**

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500,
self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

meth get_ref

```
get_ref() -> RefItem
```

get_ref.

class PictureItem

Bases: `FloatingItem`

PictureItem.

Methods:

- **caption_text** – Computes the caption as a single text.
- **export_to_doctags** – Export picture to document tokens format.
- **export_to_document_tokens** – Export to DocTags format.
- **export_to_html** – Export picture to HTML format.
- **export_to_markdown** – Export picture to Markdown format.
- **get_annotations** – Get the annotations of this PictureItem.
- **get_image** – Returns the image corresponding to this FloatingItem.
- **get_location_tokens** – Get the location string for the BaseCell.
- **get_ref** – get_ref.

Attributes:

- **annotations** (`Annotated[list[PictureDataType], deprecated('Field `annotations` is deprecated; use `meta` instead.')]`) –
- **captions** (`list[RefItem]`) –
- **children** (`list[RefItem]`) –
- **comments** (`list[FineRef]`) –
- **content_layer** (`ContentLayer`) –
- **footnotes** (`list[RefItem]`) –
- **image** (`Optional[ImageRef]`) –
- **label** (`Literal[PICTURE, CHART]`) –
- **meta** (`Optional[PictureMeta]`) –
- **model_config** –
- **parent** (`Optional[RefItem]`) –
- **prov** (`list[ProvenanceItem]`) –

- **references** (list[RefItem]) –
- **self_ref** (str) –
- **source** (Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')]) –

attr annotations

```
annotations: Annotated[list[PictureDataType], deprecated('Field `annotations` is deprecated; use `meta` instead.')] = []
```

attr captions

```
captions: list[RefItem] = []
```

attr children

```
children: list[RefItem] = []
```

attr comments

```
comments: list[FineRef] = []
```

attr content_layer

```
content_layer: ContentLayer = BODY
```

attr footnotes

```
footnotes: list[RefItem] = []
```

attr image

```
image: Optional[ImageRef] = None
```

attr label

```
label: Literal[PICTURE, CHART] = PICTURE
```

attr meta

```
meta: Optional[PictureMeta] = None
```

attr model_config

```
model_config = ConfigDict(extra='forbid')
```

attr parent

```
parent: Optional[RefItem] = None
```

attr **prov**

```
prov: list[ProvenanceItem] = []
```

attr **references**

```
references: list[RefItem] = []
```

attr **self_ref**

```
self_ref: str = Field(pattern=_JSON_POINTER_REGEX)
```

attr **source**

```
source: Annotated[list[SourceType], Field(description='The provenance of this document item. Currently, it is only used for media track provenance.')] = []
```

meth **caption_text**

```
caption_text(doc: DoclingDocument) -> str
```

Computes the caption as a single text.

meth **export_to_doctags**

```
export_to_doctags(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500, add_location: bool = True, add_caption: bool = True, add_content: bool = True)
```

Export picture to document tokens format.

Parameters:

- **doc** (`DoclingDocument`) – "DoclingDocument":
- **new_line** (`str`, default: `''`) – str (Default value = "") Deprecated
- **xsize** (`int`, default: `500`) – int: (Default value = 500)
- **ysize** (`int`, default: `500`) – int: (Default value = 500)
- **add_location** (`bool`, default: `True`) – bool: (Default value = True)
- **add_caption** (`bool`, default: `True`) – bool: (Default value = True)
- **add_content** (`bool`, default: `True`) – bool: (Default value = True)

meth **export_to_document_tokens**

```
export_to_document_tokens(*args, **kwargs)
```

Export to DocTags format.

meth **export_to_html**

```
export_to_html(doc: DoclingDocument, add_caption: bool = True, image_mode: ImageRefMode = PLACEHOLDER) -> str
```

Export picture to HTML format.

meth export_to_markdown

```
export_to_markdown(doc: DoclingDocument, add_caption: bool = True, image_mode: ImageRefMode = EMBEDDED
image_placeholder: str = '<!-- image -->') -> str
```

Export picture to Markdown format.

meth get_annotations

```
get_annotations() -> Sequence[BaseAnnotation]
```

Get the annotations of this PictureItem.

meth get_image

```
get_image(doc: DoclingDocument, prov_index: int = 0) -> Optional[Image]
```

Returns the image corresponding to this FloatingItem.

This function returns the PIL image from self.image if one is available. Otherwise, it uses DocItem.get_image to get an image of this FloatingItem.

In particular, when self.image is None, the function returns None if this FloatingItem has no valid provenance or the doc does not contain a valid image for the required page.

meth get_location_tokens

```
get_location_tokens(doc: DoclingDocument, new_line: str = '', xsize: int = 500, ysize: int = 500,
self_closing: bool = False) -> str
```

Get the location string for the BaseCell.

meth get_ref

```
get_ref() -> RefItem
```

get_ref.

class ImageRef

Bases: `BaseModel`

ImageRef.

Methods:

- `from_pil` – Construct ImageRef from a PIL Image.
- `validate_mimetype` – validate_mimetype.

Attributes:

- `dpi` (`int`) –

- **mimetype** (str) –
- **pil_image** (Optional[Image]) – Return the PIL Image.
- **size** (Size) –
- **uri** (Union[AnyUrl, Path]) –

attr **dpi**

```
dpi: int
```

attr **mimetype**

```
mimetype: str
```

attr **pil_image**

```
pil_image: Optional[Image]
```

Return the PIL Image.

attr **size**

```
size: Size
```

attr **uri**

```
uri: Union[AnyUrl, Path] = Field(union_mode='left_to_right')
```

meth **from_pil**

```
from_pil(image: Image, dpi: int) -> Self
```

Construct ImageRef from a PIL Image.

meth **validate_mimetype**

```
validate_mimetype(v)
```

validate_mimetype.

class PictureClassificationClass

Bases: BaseModel

PictureClassificationData.

Attributes:

- **class_name** (str) –
- **confidence** (float) –

attr **class_name**

```
class_name: str
```

attr **confidence**

```
confidence: float
```

class PictureClassificationData

Bases: `BaseAnnotation`

PictureClassificationData.

Attributes:

- **kind** (`Literal['classification']`) –
- **predicted_classes** (`list[PictureClassificationClass]`) –
- **provenance** (`str`) –

attr **kind**

```
kind: Literal['classification'] = 'classification'
```

attr **predicted_classes**

```
predicted_classes: list[PictureClassificationClass]
```

attr **provenance**

```
provenance: str
```

class RefItem

Bases: `BaseModel`

RefItem.

Methods:

- **get_ref** – get_ref.
- **resolve** – Resolve the path in the document.

Attributes:

- **cref** (`str`) –
- **model_config** –

attr **cref**

```
cref: str = Field(alias='$ref', pattern=_JSON_POINTER_REGEX)
```

attr **model_config**

```
model_config = ConfigDict(populate_by_name=True)
```

meth **get_ref**

```
get_ref()
```

get_ref.

meth **resolve**

```
resolve(doc: DoclingDocument)
```

Resolve the path in the document.

class BoundingBox

Bases: `BaseModel`

BoundingBox.

Methods:

- **area** – area.
- **as_tuple** – as_tuple.
- **enclosing_bbox** – Create a bounding box that covers all of the given boxes.
- **expand_by_scale** – expand_to_size.
- **from_tuple** – from_tuple.
- **get_intersection_bbox** – Return the intersection bounding box with another bounding box or `None` when disjoint.
- **intersection_area_with** – Calculate the intersection area with another bounding box.
- **intersection_over_self** – intersection_over_self.
- **intersection_over_union** – intersection_over_union.
- **is_above** – is_above.
- **is_horizontally_connected** – is_horizontally_connected.
- **is_left_of** – is_left_of.
- **is_strictly_above** – is_strictly_above.
- **is_strictly_left_of** – is_strictly_left_of.
- **normalized** – normalized.
- **overlaps** – overlaps.
- **overlaps_horizontally** – Check if two bounding boxes overlap horizontally.
- **overlaps_vertically** – Check if two bounding boxes overlap vertically.
- **overlaps_vertically_with_iou** – overlaps_y_with_iou.
- **resize_by_scale** – resize_by_scale.

- `scale_to_size` – scale_to_size.
- `scaled` – scaled.
- `to_bottom_left_origin` – to_bottom_left_origin.
- `to_top_left_origin` – to_top_left_origin.
- `union_area_with` – Calculates the union area with another bounding box.
- `x_overlap_with` – Calculates the horizontal overlap with another bounding box.
- `x_union_with` – Calculates the horizontal union dimension with another bounding box.
- `y_overlap_with` – Calculates the vertical overlap with another bounding box, respecting coordinate origin.
- `y_union_with` – Calculates the vertical union dimension with another bounding box, respecting coordinate origin.

Attributes:

- `b` (`float`) –
- `coord_origin` (`CoordOrigin`) –
- `height` – height.
- `l` (`float`) –
- `r` (`float`) –
- `t` (`float`) –
- `width` – width.

`attr` **b**

```
b: float
```

`attr` **coord_origin**

```
coord_origin: CoordOrigin = TOPLEFT
```

`attr` **height**

```
height
```

height.

`attr` **l**

```
l: float
```

`attr` **r**

```
r: float
```

`attr` **t**

```
t: float
```

`attr` **width**

```
width
```

width.

`meth` **area**

```
area() -> float
```

area.

`meth` **as_tuple**

```
as_tuple() -> tuple[float, float, float, float]
```

as_tuple.

`meth` **enclosing_bbox**

```
enclosing_bbox(boxes: list[BoundingBox]) -> BoundingBox
```

Create a bounding box that covers all of the given boxes.

`meth` **expand_by_scale**

```
expand_by_scale(x_scale: float, y_scale: float) -> BoundingBox
```

expand_to_size.

`meth` **from_tuple**

```
from_tuple(coord: tuple[float, ...], origin: CoordOrigin)
```

from_tuple.

Parameters:

- **coord** (`tuple[float, ...]`) – tuple[float:
- `...]` –
- **origin** (`CoordOrigin`) – CoordOrigin:

`meth` **get_intersection_bbox**

```
get_intersection_bbox(other: BoundingBox) -> Optional BoundingBox]
```

Return the intersection bounding box with another bounding box or `None` when disjoint.

`meth` **intersection_area_with**

```
intersection_area_with(other: BoundingBox) -> float
```

Calculate the intersection area with another bounding box.

meth intersection_over_self

```
intersection_over_self(other: BoundingBox, eps: float = 1e-06) -> float
```

intersection_over_self.

meth intersection_over_union

```
intersection_over_union(other: BoundingBox, eps: float = 1e-06) -> float
```

intersection_over_union.

meth is_above

```
is_above(other: BoundingBox) -> bool
```

is_above.

meth is_horizontally_connected

```
is_horizontally_connected(elem_i: BoundingBox, elem_j: BoundingBox) -> bool
```

is_horizontally_connected.

meth is_left_of

```
is_left_of(other: BoundingBox) -> bool
```

is_left_of.

meth is_strictly_above

```
is_strictly_above(other: BoundingBox, eps: float = 0.001) -> bool
```

is_strictly_above.

meth is_strictly_left_of

```
is_strictly_left_of(other: BoundingBox, eps: float = 0.001) -> bool
```

is_strictly_left_of.

meth normalized

```
normalized(page_size: Size)
```

normalized.

meth overlaps

```
overlaps(other: BoundingBox) -> bool
```

overlaps.

`meth` **overlaps_horizontally**

```
overlaps_horizontally(other: BoundingBox) -> bool
```

Check if two bounding boxes overlap horizontally.

`meth` **overlaps_vertically**

```
overlaps_vertically(other: BoundingBox) -> bool
```

Check if two bounding boxes overlap vertically.

`meth` **overlaps_vertically_with_iou**

```
overlaps_vertically_with_iou(other: BoundingBox, iou: float) -> bool
```

overlaps_y_with_iou.

`meth` **resize_by_scale**

```
resize_by_scale(x_scale: float, y_scale: float)
```

resize_by_scale.

`meth` **scale_to_size**

```
scale_to_size(old_size: Size, new_size: Size)
```

scale_to_size.

`meth` **scaled**

```
scaled(scale: float)
```

scaled.

`meth` **to_bottom_left_origin**

```
to_bottom_left_origin(page_height: float) -> BoundingBox
```

to_bottom_left_origin.

Parameters:

- `page_height` (`float`) –

`meth` **to_top_left_origin**

```
to_top_left_origin(page_height: float) -> BoundingBox
```

to_top_left_origin.

Parameters:

- `page_height` (`float`) –

`meth` **union_area_with**

```
union_area_with(other: BoundingBox) -> float
```

Calculates the union area with another bounding box.

`meth` **x_overlap_with**

```
x_overlap_with(other: BoundingBox) -> float
```

Calculates the horizontal overlap with another bounding box.

`meth` **x_union_with**

```
x_union_with(other: BoundingBox) -> float
```

Calculates the horizontal union dimension with another bounding box.

`meth` **y_overlap_with**

```
y_overlap_with(other: BoundingBox) -> float
```

Calculates the vertical overlap with another bounding box, respecting coordinate origin.

`meth` **y_union_with**

```
y_union_with(other: BoundingBox) -> float
```

Calculates the vertical union dimension with another bounding box, respecting coordinate origin.

`class` CoordOrigin

Bases: `str`, `Enum`

CoordOrigin.

Attributes:

- `BOTTOMLEFT` –
- `TOPLEFT` –

`attr` **BOTTOMLEFT**

```
BOTTOMLEFT = 'BOTTOMLEFT'
```

`attr` **TOPLEFT**

```
TOPLEFT = 'TOPLEFT'
```

class ImageRefMode

Bases: `str`, `Enum`

ImageRefMode.

Attributes:

- **EMBEDDED** –
- **PLACEHOLDER** –
- **REFERENCED** –

attr EMBEDDED

```
EMBEDDED = 'embedded'
```

attr PLACEHOLDER

```
PLACEHOLDER = 'placeholder'
```

attr REFERENCED

```
REFERENCED = 'referenced'
```

class Size

Bases: `BaseModel`

Size.

Methods:

- **as_tuple** – as_tuple.

Attributes:

- **height** (`float`) –
- **width** (`float`) –

attr height

```
height: float = 0.0
```

attr width

```
width: float = 0.0
```

meth as_tuple

```
as_tuple()
```

as_tuple.